SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

---

COURSE OUTCOME COVERED

CO4: Examine the applicability of clustering algorithms in real-world scenarios, presenting findings through clear reports with effective visualizations.
Assessment Tool Weightage: 20%

QUESTIONS: 05 **Total Marks:50**

Annexure A

COMPARITIVE ANALYSIS

Criteria	Scope of Items *(2)*	Comparison Depth(2.5)	Use of Evidence *(2)*	Critical Thinking *(2)*	Clarity of Presentation *(1)*	***Total(10)***
Weightage	*20%*	25%	20%	20%	10%	*100%*
Q1						
Q2						
Q3						
Q4						
Q5						

Code of Conduct:

I A. LOKESH KUMAR (192524157)) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

		Comparative analysis			
Criteria	**Weightage (%)**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Annexure A

Q1 . Dataset: Assessment Tool 3: Comparative Analysis (25% → 5 Marks)

Time duration :45 Minutes
Assessment Tool 3 (Low)

Q1 . Dataset: Customer Data

Customer	Income (₹)	Spending Score
C1	30000	60
C2	60000	40
C3	50000	70
C4	28000	65
C5	65000	35

Question:
Compare K-means and hierarchical clustering using this dataset and explain differences in performance and scalability. *(BL3 – 1 Mark)*

Q2. Dataset: Document Data

Document	Word1	Word2	Word3
D1	0.2	0.5	0.3
D2	0.1	0.6	0.3
D3	0.8	0.1	0.1
D4	0.75	0.15	0.1
D5	0.2	0.4	0.4

Question:
Compare K-means and EM algorithms in handling overlapping clusters for the given dataset. *(BL4 – 1 Mark)*

Q3. Dataset: Geographic Data

Point	Latitude	Longitude
P1	13.08	80.27
P2	13.10	80.25
P3	12.97	80.22
P4	13.05	80.30
P5	13.00	80.20

Question:
Compare Euclidean and Manhattan distance metrics and analyze their impact on clustering results. *(BL4 – 1 Mark)*

Q4. Dataset: Retail Products

Product	Sales (₹)	Frequency
P1	50000	20
P2	30000	10
P3	52000	22
P4	25000	8
P5	32000	12

Question:
Compare agglomerative and divisive clustering approaches and explain their suitability for this dataset. *(BL3 – 1 Mark)*

Q5 . Dataset: Mixed Attributes

Item	Price (₹)	Category	Rating
I1	500	A	4.5
I2	300	B	3.8
I3	550	A	4.7
I4	250	B	3.5
I5	520	A	4.6

Question:
Compare centroid-based and density-based clustering techniques and analyze their effectiveness on mixed data types. *(BL4 – 1 Mark)*

Answer 1

Dataset Overview

The Customer Data dataset contains five customers (C1–C5) described by two attributes: Income (₹) and Spending Score. C1 (₹30,000, score 60) and C4 (₹28,000, score 65) represent a lower-income, high-spending-score group; C2 (₹60,000, score 40) and C5 (₹65,000, score 35) represent a higher-income, lower-spending-score group; and C3 (₹50,000, score 70) sits with moderate income but the highest spending score. This small, well-separated dataset is a convenient testbed for comparing how K-means and hierarchical clustering behave.

Comparison of K-Means and Hierarchical Clustering

- Algorithmic approach: K-means is a partitional, centroid-based algorithm that iteratively assigns each point to the nearest of K centroids and recomputes centroids until convergence. Hierarchical (agglomerative) clustering instead builds a nested tree of clusters bottom-up, starting with each point as its own cluster and successively merging the closest pairs until one cluster remains, visualized as a dendrogram.
- Number of clusters: K-means requires K to be specified in advance (e.g., using the elbow method to test K=2 on this data, which separates the low-spending {C2, C5} pair from the high-spending {C1, C3, C4} group). Hierarchical clustering does not require K upfront — the dendrogram can be cut at any height afterward to obtain 2, 3, or more clusters, offering more flexibility for this small dataset where the 'right' K is not obvious in advance.
- Performance/accuracy on this data: Both algorithms would likely produce similar groupings here because the data is small and the two behavioural groups (high-income/low-spending vs low-income/high-spending) are fairly well separated in the Income–Spending Score space. K-means, being sensitive to initial centroid placement, may occasionally produce a slightly different split on a small dataset like this if centroids are seeded poorly, whereas hierarchical clustering is deterministic — the same input always produces the same dendrogram.
- Scalability: K-means is far more scalable, with a time complexity of roughly $O(n \cdot K \cdot i)$ per iteration (n points, K clusters, i iterations), making it practical for large customer databases with thousands or millions of records. Hierarchical clustering has a time complexity of roughly $O(n^2)$ or worse (due to repeated distance-matrix computation and updates), which becomes computationally expensive very quickly as the number of customers grows — a critical consideration if this five-row sample were scaled up to a real retail customer base.
- Interpretability: Hierarchical clustering's dendrogram gives a rich, visual, multi-level view of how customers relate to each other at every possible cut level, which is valuable for exploratory analysis of a small customer segment. K-means gives only a single flat partition for a chosen K, offering less structural insight but a faster, more direct answer once K is decided.

In summary, for this small five-customer dataset both algorithms would likely converge to a similar two-group split (high-income/low-spending versus low-income/high-spending), but K-means is the more practical choice for large-scale, repeated customer segmentation due to its lower computational cost and better scalability, while hierarchical clustering is more useful in this exploratory context for visually confirming the natural number of clusters via the dendrogram before committing to a K for a faster algorithm like K-means in production.

Answer 2

Dataset Overview

The Document Data dataset represents five documents (D1–D5) as vectors of three word-weight features (Word1, Word2, Word3), typical of a simplified TF-IDF-style representation. D1 (0.2, 0.5, 0.3) and D2 (0.1, 0.6, 0.3) are very similar, both dominated by Word2. D3 (0.8, 0.1, 0.1) and D4 (0.75, 0.15, 0.1) are similar to each other, both dominated by Word1. D5 (0.2, 0.4, 0.4) is close to the D1/D2 pair but has a noticeably higher Word3 weight, placing it in a position that could plausibly be shared between the Word2-dominant group and a hypothetical Word3-dominant theme — a good illustration of an overlapping or ambiguous document.

Comparison of K-Means and EM (Expectation-Maximization / Gaussian Mixture) for Overlapping Clusters

- Cluster assignment model: K-means performs hard assignment — every document is assigned to exactly one cluster based on nearest centroid (Euclidean distance), forcing a document like D5, which sits between two thematic groups, into a single cluster even though it shares characteristics with both. The EM algorithm (typically used to fit a Gaussian Mixture Model) performs soft assignment — it computes a probability that each document belongs to each cluster, so D5 could be represented as, for example, 65% belonging to the Word2-dominant topic and 35% belonging to a Word3-leaning topic, capturing its genuinely ambiguous nature.
- Cluster shape assumptions: K-means implicitly assumes roughly spherical, equally-sized clusters because it minimizes within-cluster variance using Euclidean distance. EM/GMM can model elliptical clusters of varying size and orientation by fitting a full covariance matrix per cluster, which better accommodates real text data where topics often have different spreads across vocabulary dimensions.
- Handling of overlapping regions: For documents like D5 that lie in a genuinely ambiguous region between two topics, K-means' hard boundary can misclassify or arbitrarily assign the point to whichever centroid happens to be marginally closer, which can be unstable to small changes in the data. EM naturally represents this ambiguity through its probability distribution, which is more faithful to the reality that many real documents discuss multiple topics simultaneously.
- Computational cost: K-means is computationally cheaper and converges faster, since it only requires distance calculations and centroid updates. EM is more computationally intensive, requiring repeated likelihood computations and covariance matrix updates in the M-step, but this additional cost is often justified for text/topic data where overlapping themes are common and a hard partition would lose important information.
- Output usefulness for document clustering specifically: For applications such as topic modelling or document recommendation, EM's soft membership scores are directly useful — a document like D5 can be shown to users interested in either topic, weighted by its membership probability, whereas K-means would show it only to users of a single, possibly arbitrarily chosen topic.

In summary, K-means would likely produce two clean clusters — {D1, D2, D5} (Word2-dominant) and {D3, D4} (Word1-dominant) — by forcing D5 into the nearest group, whereas EM would additionally reveal that D5 is a genuinely mixed-membership document with meaningful partial similarity to both groups. For this dataset, EM is the more appropriate technique whenever the overlapping nature of D5's word-weight profile is analytically

important, while K-means remains preferable when a fast, simple, hard partition is sufficient for the task at hand.

Answer 3

Dataset Overview

The Geographic Data dataset contains five points (P1–P5) with Latitude and Longitude coordinates, all clustered closely around the Chennai region (roughly 13.0°N, 80.2–80.3°E). P1 (13.08, 80.27), P2 (13.10, 80.25), and P4 (13.05, 80.30) are relatively close together in the northern part of the range, while P3 (12.97, 80.22) and P5 (13.00, 80.20) are closer together slightly to the south-west. This dataset is well suited to illustrating how the choice of distance metric affects clustering, since the points differ from each other along both the latitude and longitude axes by varying amounts.

Comparison of Euclidean and Manhattan Distance Metrics

- Mathematical definition: Euclidean distance between two points computes the straight-line ('as the crow flies') distance, using the square root of the sum of squared coordinate differences: $\sqrt{[(\Delta lat)^2 + (\Delta lon)^2]}$. Manhattan distance (also called city-block or L1 distance) instead sums the absolute coordinate differences along each axis separately: $|\Delta lat| + |\Delta lon|$, resembling travel along a rectangular grid of streets rather than a direct line.
- Effect on this geographic data: For points like P1 (13.08, 80.27) and P2 (13.10, 80.25), the Euclidean distance is $\sqrt{[(0.02)^2 + (0.02)^2]} \approx 0.028$, while the Manhattan distance is $|0.02| + |0.02| = 0.04$. Because Manhattan distance never takes the shorter diagonal path that Euclidean distance allows, it will generally report larger distances between diagonally offset points, which can shift which points are judged 'closest' when differences are similar in both the latitude and longitude directions.
- Sensitivity to directionality: Euclidean distance treats deviation in any direction equally and rewards points that are close in a straight line, making it more appropriate when actual geographic proximity (e.g., straight-line distance for a delivery drone) is what matters. Manhattan distance is more appropriate when movement is constrained to a grid-like structure (e.g., city streets laid out in a grid, or when latitude and longitude differences represent genuinely independent, non-interchangeable costs, such as separate road segments running north-south and east-west).
- Impact on cluster shape: Euclidean distance tends to produce roughly circular (spherical, in higher dimensions) clusters, since it measures distance uniformly in all directions. Manhattan distance tends to produce more diamond- or square-shaped cluster boundaries, since it privileges movement along the coordinate axes over diagonal movement — this can noticeably change which points at the boundary between two nearby clusters (such as P4 and P1, or P3 and P5) end up grouped together.
- Robustness considerations: Manhattan distance is generally less sensitive to outliers than Euclidean distance, because it does not square the coordinate differences (which would otherwise disproportionately amplify large deviations). For geographic data that might include an occasional erroneous or extreme coordinate reading, Manhattan distance can therefore produce more stable clustering results.

In summary, applying Euclidean distance to this dataset would likely group points based on their true straight-line geographic proximity — for example, pairing P3 and P5 together as the closest true neighbours — while Manhattan distance could shift some borderline pairings (such as P1, P2, and P4) depending on how their latitude and longitude differences compare

individually rather than combined diagonally. For genuine geographic proximity analysis such as identifying real-world nearby locations, Euclidean distance is generally the more natural and widely used choice, while Manhattan distance remains useful when the underlying movement or cost structure is inherently grid-based rather than direct-line.

Answer 4

Dataset Overview

The Retail Products dataset contains five products (P1–P5) described by Sales (₹) and Frequency (number of times purchased). P1 (₹50,000, 20) and P3 (₹52,000, 22) form a high-sales, high-frequency group, while P2 (₹30,000, 10), P4 (₹25,000, 8), and P5 (₹32,000, 12) form a lower-sales, lower-frequency group. This creates a reasonably clear two-cluster structure that is useful for comparing how agglomerative and divisive clustering would approach the same data from opposite directions.

Comparison of Agglomerative and Divisive Clustering

- Direction of construction: Agglomerative clustering is a bottom-up approach — it starts with each of the five products as its own individual cluster and progressively merges the two closest clusters at each step (for example, P1 and P3 would merge first due to their very similar Sales and Frequency values, followed by P2, P4, and P5 merging among themselves) until all products form a single cluster. Divisive clustering is a top-down approach — it starts with all five products in one cluster and repeatedly splits the least cohesive cluster into two, continuing recursively until each product is its own cluster or a stopping criterion is met.
- Computational cost: Agglomerative clustering is generally more efficient and far more commonly used in practice, since at each step it only needs to identify the closest pair of existing clusters to merge — a manageable $O(n^2)$ or $O(n^2 \log n)$ operation for a small dataset like this five-product sample. Divisive clustering is considerably more expensive, because splitting a cluster optimally at each step is itself a combinatorial sub-problem (deciding the best way to partition, for example, all five products, then the best way to partition each resulting subgroup), making it impractical for anything beyond very small datasets.
- Quality of early decisions: In agglomerative clustering, early merge decisions (such as merging P1 and P3, the two clearly similar high-performing products) are usually reliable because they are based on genuinely close pairs, but any mistake made early on cannot be undone later. In divisive clustering, the very first split — dividing all five products into two groups — is the most difficult and most consequential decision, since it must correctly identify the primary two-group structure (high-performing {P1, P3} versus lower-performing {P2, P4, P5}) without the benefit of having first resolved any smaller-scale similarities.
- Suitability for this dataset: Given the small size of this dataset (five products) and the relatively clear two-group separation between high-performing and lower-performing products, agglomerative clustering is more suitable and more practical: it will efficiently and reliably merge P1 with P3 and P2, P4, and P5 among themselves based on their genuine pairwise similarity, producing an interpretable dendrogram at low computational cost. Divisive clustering would arrive at a similar final structure in this simple case but at a much higher computational cost that is not justified for a dataset this small and this cleanly separated.

In summary, while both agglomerative and divisive clustering could ultimately reveal the same underlying two-group structure in this retail dataset — separating high-sales, high-frequency products (P1, P3) from lower-sales, lower-frequency products (P2, P4, P5) — agglomerative clustering is the more suitable and standard choice in practice due to its lower computational complexity, its reliance on well-defined, easily computed pairwise similarities, and its wide support in standard clustering libraries, whereas divisive clustering is rarely used outside of specialized applications where a genuinely top-down, whole-to-parts view of the data is specifically required.

Answer 5

Dataset Overview

The Mixed Attributes dataset contains five items (I1–I5) described by Price (₹), Category (a categorical attribute with values A or B), and Rating (a continuous score). I1 (₹500, A, 4.5), I3 (₹550, A, 4.7), and I5 (₹520, A, 4.6) form a tight, high-price, high-rating group all within Category A. I2 (₹300, B, 3.8) and I4 (₹250, B, 3.5) form a distinct lower-price, lower-rating group within Category B. Notably, the dataset mixes a numeric attribute (Price, Rating) with a categorical attribute (Category), which directly affects how centroid-based and density-based clustering techniques can be applied.

Comparison of Centroid-Based and Density-Based Clustering on Mixed Data

- Core mechanism: Centroid-based clustering (e.g., K-means) represents each cluster by the mean (centroid) of its members' numeric attributes and assigns points to the nearest centroid. Density-based clustering (e.g., DBSCAN) instead groups together points that are closely packed (high density) in the feature space, and marks points in low-density regions as noise/outliers, without requiring the number of clusters to be specified in advance.
- Handling the categorical attribute (Category): Centroid-based methods like K-means are designed for numeric data and cannot directly compute a meaningful 'average' of a categorical attribute like Category (A or B); Category must first be encoded (e.g., one-hot encoded into numeric indicator columns) before it can be included, and even then the resulting Euclidean distances treat the encoded category somewhat arbitrarily rather than reflecting true categorical similarity. Density-based methods face a similar encoding requirement for standard Euclidean-based DBSCAN, though density-based approaches can more naturally be adapted with a custom or mixed-type distance metric (e.g., Gower's distance) that handles categorical and numeric attributes together more principledly.
- Cluster shape assumptions: K-means assumes roughly spherical clusters of similar size and works best when clusters are compact and well-separated by simple distance, which fits this dataset reasonably well since the two natural groups (I1, I3, I5) and (I2, I4) are compact in Price-Rating space. DBSCAN can discover clusters of arbitrary shape and does not require the number of clusters to be pre-specified, which is advantageous if the true grouping structure were more irregular than the clean two-group split seen here, but with only five points DBSCAN's density parameters (minPts, epsilon) are hard to tune reliably.
- Effectiveness on this specific dataset: Given that Category A items (I1, I3, I5) are also the higher-price, higher-rating items, and Category B items (I2, I4) are also the lower-price, lower-rating items, the categorical attribute and the numeric attributes are strongly aligned here — meaning centroid-based K-means on just Price and Rating (or with a simply-encoded Category) would already recover the correct two-group structure effectively, without needing DBSCAN's more flexible density-based approach. DBSCAN would also

succeed on this clearly separated data but offers little additional benefit here, since its main strengths (arbitrary cluster shapes, automatic outlier detection, no need to pre-specify K) are not particularly needed for such a small, cleanly separated dataset.

- Outlier sensitivity: If this dataset were extended with a genuinely anomalous item (e.g., a very high-priced Category B item with a poor rating), DBSCAN would naturally flag it as noise rather than forcing it into one of the two main clusters, while K-means would be forced to assign it to whichever centroid is nearest, potentially distorting that cluster's centroid and reducing the quality of the overall clustering.

In summary, for this cleanly separated, mixed-attribute dataset where the categorical Category attribute aligns closely with the numeric Price and Rating values, centroid-based clustering (K-means) is effective and computationally simple, provided Category is appropriately encoded. Density-based clustering (DBSCAN) becomes more valuable as data complexity increases — with larger datasets, irregularly shaped clusters, or a genuine need for automatic outlier detection — but offers limited additional benefit over K-means for a dataset as small and well-structured as this one.